

Claude Sonnet 5.5 across every effort level

VulcanBench Frontier v4 | Code quality protocol v3.23 | October 2026

Abstract

Claude Sonnet 5.5 ran the 23-task VulcanBench Frontier v4 suite through Claude Code 2.1.291 to 2.1.293 on a Claude Max subscription from October 6 to 8, 2026, once per task at each of five effort levels, Low to Max, 115 runs in all, one task at a time on the owner's Mac. Code quality carries 33% of the combined score and is judged for a named human reader by Muse Spark 1.3 (Meta) and Grok 4.6 (xAI) under the same rubric, controls and gates as the other Frontier v4 reports, with a ground-truth intent-recovery probe. The combined score is 81.63 at Low, 84.10 at Medium, 86.79 at High, 90.20 at Extra-high and 92.02 at Max. Sonnet 5.5 passes 15, 17, 20, 22 and 23 of 23 tasks from Low to Max and fixes 214, 219, 223, 230 and 231 of the 231 hidden behaviours tested. Code quality runs from 61.67 to 81.72. Cost per task, as Claude Code reports it, runs from \$2.39 to \$5.52; the whole sweep comes to \$394.77, \$3.43 per task.

Every run finished inside the flat 3-hour task bound and none was excluded; every cell is judged on 23 runs, each the equal mean of both judges. Claude Code's refusal fallback was on and no run used it. Two facts about how this column was produced are disclosed throughout: the sweep predates the harness's tagged-worktree rule and was admitted to judging through a committed task hash bridge, and Grok 4.6 judged through a newer Cursor CLI than in earlier rounds.

Table 1. Combined score, Code quality, runtime and tasks passed by effort

Effort	Combined (33%)	SE	Combined (20%)	Code quality	Min/task	Judged	Passed
Low	81.63	1.38	85.24	61.67	20.0	23	15/23
Medium	84.10	1.20	87.25	65.35	20.8	23	17/23
High	86.79	1.15	89.29	70.84	14.8	23	20/23
Extra-high	90.20	0.57	92.08	75.78	21.6	23	22/23
Max	92.02	0.35	92.91	81.72	38.0	23	23/23

Combined (33%) = 0.50 functional + 0.085 lint and complexity + 0.085 security + 0.33 Code quality. Combined (20%) applies the prior 50/15/15/20 profile to the same Code quality scores. SE is one sample standard error across the cell's tasks, not judge uncertainty or a significance test. Runtime is solver wall-clock per task; judging is excluded. Frontier Code quality is the reviewed layer plus intent recovery and is never compared with Routine v1 Code quality.

The Code quality protocol

Two failure modes motivated the reviewed score. The lint and complexity metric rewards compression: the maintainability index carries a lines-of-code term and per-function complexity stays low when each dense line does something different, so five statements on a line with names like *x* and *k* can outscore the same logic written for a person. And a large model reading compressed code pays almost nothing to parse it; when a frontier model was calibrated as a readability judge, it rated a squashed six-line function the same as its formatted copy. Since September 7, 2026 Code quality carries 33% of the combined score and lint and complexity and security carry 8.5% each. The prior profile is reported beside the new one on every table.

Protocol v3.23 is the v3.15 protocol applied to this population. Nothing in the rubric, controls, quirk keys, gates, repeats, seed, weights or judge settings changed; both judges retook the calibration exam under v3.23 before any counted call. What changed is which Cursor CLI carried Grok 4.6, described on the next page, and how runs were admitted, described under How it was run. The population froze on October 8 with 115 rows, none missing and none excluded.

The rubric

Every judge receives the same instructions, frozen by hash before any review. The prompt names the reader it scores for: an engineer who has never seen the code, reads it top to bottom without running it, and must make a correct change in one sitting. It tells the judge that its own ease at parsing dense code is not evidence of readability. Six dimensions are scored 0 to 4 in half steps against written anchors, each with an exact excerpt and a concrete consequence for that reader.

Sub-score	Dimension	What is assessed
Human readability	Naming	Identifiers say what things are in the task's domain
Human readability	Presentation	Statements per line, nesting, function length, how much the reader must hold at once
Human readability	Intent	Constants, thresholds and quirks explained by names or accurate comments that say why
Maintainability	Structure	Coherent responsibilities, no duplicated policy, no abstraction beyond the task's scale
Maintainability	Changeability	A named plausible change, every edit site traced, scored on locality and safety
Maintainability	Verifiability	Explicit state, failures that name what went wrong, seems to test one rule alone

The layers under the 33 points

Reviewed panel, 24 points. The six-dimension rubric above, averaged across the judges with a valid review. **Intent recovery, 9 points.** Every task in this suite is a rewrite of a retired binary whose real behaviour departs from its written specification in documented ways. The judge sees only the specification and the code, never the issue text, and lists where the code departs from the specification; a separate call matches that list against the frozen answer key. The denominator is the quirks the submission actually passed tests for, so a functional failure is not punished twice; a submission that passed none has no intent-recovery score and its Code quality is the reviewed score alone, which no Sonnet 5.5 run needed. **Measured maintenance, designed for 12 points, not yet built.** Until it exists the pre-registered split above is in force.

Judges and calibration

The scored panel is Muse Spark 1.3 through the Muse CLI on its Standard tier, which does not train on prompts, and Grok 4.6 through the Cursor CLI at medium effort. Neither lab has a model on this board, so neither judge grades a relative, and both are neutral for an Anthropic submission. Every session is fresh, tools are disabled, the workspace is empty, model identity is checked per call from the CLI's own records, and solver labels are withheld.

Judge settings and versions

The original v3.3 and v3.4 judge protocol files were recovered from the owner's private backup, and their sha256 equal the published values (v3.3 b82da598..., v3.4 1d80e097...). The judge settings used in v3.23, read from docs/judging/judge-pins-v3.json, are identical to them. Muse Spark 1.3 ran the same binary as every earlier round (sha256 4c0f960028b6..., a match). Grok 4.6 ran on a newer Cursor CLI: the Cursor pin hashes only Cursor's launcher script, which is the same in every Cursor release, so it never fixed the version. Cursor updated itself on October 7 at 14:43 PDT, and every v3.23 Grok call ran on Cursor CLI 2026.10.01-e373342, while the v3.3 round recorded 2026.09.02-c22c1a3. The Grok model id is the same; Cursor reports its display name as Grok 4.6 Medium where the frozen settings say Cursor Grok 4.6 Medium, a rename the judging wrapper accepts and records. Grok 4.6 passed calibration under v3.23.

The calibration exam

Before scoring a single submission each judge reviews ten held-out programs implementing one ledger specification, from clear to compressed, misleadingly commented, narrated, needlessly abstracted and carrying an embedded instruction to give full marks, five times each in a seeded order. Twenty gates fixed in advance check that the judge sees each construct; a judge may miss at most one gate by at most half a point.

Table 2. Calibration verdicts

Judge	Protocol	Calls	Result	Allowance	Failing gates
Muse Spark 1.3	code-quality-maintenance-v3.23	80	passed	used	g11_repeatability (short by 0.02)
Grok 4.6	code-quality-maintenance-v3.23	80	passed	used	g04_formatting_is_presentation (short by 0.10)

Both judges passed, each using the one-gate allowance, both well inside the half-point limit. Under v3.15, for Opus 5.5, both passed every gate with no allowance used.

Table 3. Control means on three of the ten calibration programs

Judge	Control	Naming	Present.	Intent	Struct.	Change.	Verif.
Muse	clear	4.00	3.70	3.60	4.00	4.00	3.70
Muse	compressed	1.30	1.90	1.10	3.60	3.00	3.00
Muse	misleading comments	4.00	3.70	0.70	4.00	2.40	3.50
Grok	clear	3.60	3.60	3.50	4.00	4.00	3.50
Grok	compressed	1.50	1.50	1.50	3.10	2.40	3.00
Grok	misleading comments	3.30	3.30	1.20	3.60	2.40	3.10

All ten controls, every gate value and both judges' control means are in calibration.json.



Results in detail

Table 4. Code quality components at each effort level

Component	Low	Medium	High	Extra-high	Max
Code quality	61.67	65.35	70.84	75.78	81.72
Human readability	53.4	54.4	65.9	72.2	79.5
Maintainability	64.9	68.7	75.1	78.0	84.1
Intent recovery	68.4	75.5	71.8	77.6	81.4
Reviewed score	59.1	61.5	70.5	75.1	81.8
Rated by Muse Spark 1.3	56.1	60.4	69.7	74.4	81.5
Rated by Grok 4.6	62.2	62.7	71.3	75.8	82.2
Standard error of Code quality	2.08	1.66	1.57	1.28	0.99
Hidden behaviours fixed of 231	214	219	223	230	231

Means cover the judged runs per cell (n in Table 1). Per-judge means cover the runs each judge reviewed validly.

Table 5. Four factors by effort, mean score out of 100

Effort	Functional	Lint, complexity	Security	Code quality
Low	92.13	78.97	100.00	61.67
Medium	94.59	79.22	100.00	65.35
High	96.21	80.14	100.00	70.84
Extra-high	99.71	81.99	98.48	75.78
Max	100.00	83.17	93.91	81.72

Refusal fallback

Claude Code's refusal fallback stayed at its default (on), as for Opus 5.5, following Artificial Analysis's Default Fallback convention. No run used it: every assistant reply in every run came from claude-sonnet-5-5, and no other model appears in any run's final usage record.

Cost and tokens across effort levels

Cost is Claude Code's own list-price total for each run, checked against the session's final total, as for the Opus 5.5 column. Anthropic's pricing page lists Sonnet 5.5 at \$2 input and \$10 output per million tokens, with cache writes at \$2.50 (5 minutes) and \$4 (1 hour), but gives two cache-read prices: \$0.20 per million in its table and 0.05x input (\$0.10) in its prompt caching section. Rather than pick one, the report uses the cost Claude Code reports. Sonnet 5.5 ran on a Claude Max subscription, so these are API-equivalent estimates rather than bills; judging is excluded.

Table 6. Cost, raw tokens and runtime by effort

Effort	Runs	\$/task	SE	Level total	Tokens/task	Min/task
Low	23	\$2.88	\$0.97	\$66.15	9.02M	20.0
Medium	23	\$3.00	\$1.02	\$69.05	9.49M	20.8
High	23	\$2.39	\$0.71	\$54.98	6.77M	14.8
Extra-high	23	\$3.38	\$0.79	\$77.63	8.94M	21.6
Max	23	\$5.52	\$0.84	\$126.95	14.11M	38.0
Full sweep	115	\$3.43		\$394.77	1,112M	44.1 h

Tokens are Claude Code's session usage from its final record: input, cache reads, cache writes and output.

Limitations recorded with the estimates

- Cost is Claude Code's own list-price total for each session (`cli_reported_cost_usd`), checked against the stream's final `total_cost_usd`. Every reply came from `claude-sonnet-5-5` and no other model appears in any run's usage.
- Anthropic's pricing page lists Sonnet 5.5 cache reads at \$0.20 per million tokens in its table and at 0.05x input (\$0.10) in its prompt caching section. The export does not re-price runs at either rate; if Anthropic corrects the page, the CLI-reported figure stays the published one unless the owner decides otherwise (`harness DECISIONS.md`, 2026-10-07).
- Claude Code prices cache writes by their observed TTL; the export takes its total as reported.
- No Batch, Fast or priority pricing is applied.

Beside Claude Opus 5.5 and Claude Fable 5.1

Opus 5.5 and Fable 5.1 ran the same 23 tasks through Claude Code in September and were judged by the same judges under v3.15 and v3.4 (Opus 5.5 at High judged on 22 of 23). Their numbers are the published rows; nothing was re-judged. The three columns ran on different Claude Code versions (2.1.291 to 2.1.293, 2.1.280, and 2.1.259 to 2.1.261), so small gaps between them are harness confounded, and under v3.23 Grok 4.6 ran on a newer Cursor CLI.

Table 7. Sonnet 5.5, Opus 5.5 and Fable 5.1 by effort

Effort	Combined	Passed of 23	Code quality	\$/task	Min/task
Low	81.63 / 86.36 / 89.12	15 / 17 / 19	61.7 / 72.5 / 78.4	\$2.88 / \$1.70 / \$7.96	20.0 / 13.7 / 26.9
Medium	84.10 / 90.86 / 90.13	17 / 23 / 20	65.4 / 77.3 / 80.1	\$3.00 / \$2.84 / \$9.19	20.8 / 18.6 / 31.7
High	86.79 / 91.11 / 90.07	20 / 22 / 20	70.8 / 78.0 / 80.9	\$2.39 / \$3.27 / \$9.70	14.8 / 21.4 / 28.8
Extra-high	90.20 / 90.68 / 90.75	22 / 22 / 22	75.8 / 80.0 / 81.9	\$3.38 / \$4.11 / \$14.49	21.6 / 20.7 / 38.8
Max	92.02 / 90.22 / 91.84	23 / 21 / 23	81.7 / 79.8 / 82.4	\$5.52 / \$8.75 / \$9.06	38.0 / 37.5 / 27.1

Each cell reads Sonnet 5.5 / Opus 5.5 / Fable 5.1. Combined scores and Code quality are over judged runs; pass counts, cost and minutes cover all 23 runs per cell. Cost is Claude Code's own total for Sonnet 5.5 and Opus 5.5 and the published v3.4 figure for Fable 5.1.

On the Frontier v4 board

Sonnet 5.5's best level, Max at 92.02 (SE 0.35), ranks 4 of the board's 58 columns, behind only Grok 4.7's top three levels, which are judged by a different panel (Muse Spark 1.3 and GPT-6.1 Sol). Among columns judged by Muse Spark 1.3 and Grok 4.6 it is the highest point estimate: 0.18 above Fable 5.1 at Max (91.84, SE 0.47), within one standard error, and 0.91 above Opus 5.5 at High (91.11, SE 0.57), an edge rather than a clear win. Its levels rank 39, 35, 27, 11, 4 from Low to Max.

Perfect hidden tests at Max

At Max every task passes every hidden test, 23 of 23 and all 231 tested behaviours, so Frontier v4 no longer separates Sonnet 5.5 at Max on correctness; Code quality, the lint and security scans, and cost still do. The integrity audit is clean on all 115 runs (no web access, no benchmark-data or answer-key paths).

How it was run

Before the tagged-worktree rule. The sweep was launched on October 6 from a checkout that predates the harness's October 5 rule that every sweep runs from a tagged worktree. Run summaries therefore carry no source block, and their recorded task hashes use an older format that counted `__pycache__` files. Task content was verified identical to the frozen suite lock, and judging admitted the runs through the committed hash bridge docs/judging/task-hash-bridge-sonnet55.json (23 tasks), which pairs each recorded hash with its lock hash. All 107 cached .pyc files an agent could see were byte-identical to compiling the starting source. **Claude Code versions.** Low ran 21 tasks on 2.1.291 and 2 on 2.1.292; Medium and High on 2.1.292; Extra-high 22 on 2.1.292 and paddockcore on 2.1.293; Max on 2.1.293. Earlier Claude columns used older releases. **Where and how.** --billing subscription on a Claude Max plan, the local sandbox on the owner's Mac, a flat 3-hour task timeout, one task at a time, from October 6, 06:49 PDT, to October 8, 03:38 PDT. Anthropic's default effort for Sonnet 5.5 on the Claude API is High; the vendor does not state a Claude Code default. **One attempt per task and level;** no run was retried, and effort labels are Claude Code's own.

One attempt per task and level. The solver sweep ran October 6, 06:49 PDT, to October 8, 03:38 PDT, serially, on the owner's Mac with the local sandbox and a flat 3-hour task bound. Judging ran October 8, 05:02 to 18:02 PDT; the Claude Haiku 5.5 solver sweep, which started at 03:40 PDT after the last Sonnet 5.5 run had finished, ran on the same machine during judging, which does not touch Sonnet 5.5's runtime figures.

Evidence and reproduction

Public files: runs.json and runs.csv with every run's factors, hidden behaviours fixed, integrity verdicts, task hash bridge pair, Claude Code version, replies by serving model and each scored judge's six-dimension sub-scores and intent recovery; groups.json; calibration.json with every gate value and control mean; judge-protocols.json with the exact rubric, system text, probe and match instructions, schemas, weights and the judge settings and version record; economics.json with cost and token aggregates, the list rates as published and pricing limitations; and provenance.json with source hashes and limits.

<https://github.com/morganlinton/VulcanBenchCOM/tree/main/assets/data/swe-v4-sonnet55-v323>

Withheld: raw prompts and responses; submitted patches and reconstructed sources; quirk answer keys (they describe hidden-test behaviour); reviewer session identifiers and usage receipts; host binary paths.

Recompute the published numbers

Each run's Code quality is $(0.24 \times \text{reviewed score} + 0.09 \times \text{intent recovery}) / 0.33$, where both terms are the equal mean of the judges with a valid review. The combined score is $100 \times (0.50 F + 0.085 Q + 0.085 S + 0.33 C / 100)$ with F, Q, S on a 0 to 1 scale. Group means weight the cell's tasks equally. The export script recomputes every row from the frozen summary, checks every run's cost against Claude Code's stream, every task hash against the bridge and the suite lock, and the reviewer settings against the v3.4 bundle, and refuses to write if any value differs.

What this report does not claim

No human rated anything; the scores are model judgment for a human reader, not human validation. Standard errors describe task sampling only. The measured-maintenance layer is unbuilt, so the 33 points are 24 reviewed plus 9 intent recovery. Runs were not repeated. Hash checks bind the export to frozen files; they do not prove that judges were unbiased, that the newer Cursor CLI serves Grok 4.6 exactly as the earlier one did, or that no training overlap exists.

Appendix. Per task at Low, Medium and Max

Task	Low CQ	Max CQ	Low combined	Medium combined	Max combined
blendcore	69.1	78.3	88.39	85.54	90.79
cellarcore	47.4	83.4	80.45	78.93	91.83
codeccore	68.2	86.9	67.87	74.37	94.29
depotcore	42.5	70.6	78.90	86.59	88.21
freightcore	71.2	83.3	88.90	88.59	93.11
granarycore	40.2	75.3	76.94	78.72	88.93
hedgcore	73.4	86.1	89.78	88.85	93.99
lodgcore	55.5	76.1	83.14	87.48	90.11
matchcore	70.5	81.8	77.23	86.31	92.42
metercore	62.7	86.1	86.11	87.76	93.55
pacecore	68.1	89.2	87.89	72.33	94.33
paddockcore	46.3	75.2	76.06	75.80	89.36
payrollcore	58.1	85.2	74.27	86.95	92.71
qlite	78.8	85.6	91.41	90.87	92.98
queuecore	56.8	82.0	71.60	88.12	91.96
quotacore	64.4	80.3	86.84	87.60	92.34
reflowcore	60.6	77.6	73.15	74.53	90.64
schedcore	63.3	85.7	86.17	87.35	93.01
settlecore	68.4	82.6	77.70	86.43	91.99
snapcore	64.1	86.6	86.70	86.23	93.56
stampcore	63.3	80.8	86.38	89.83	92.54
tallycore	58.5	76.4	84.85	87.61	90.86
vaultcore	67.0	84.3	76.67	77.41	92.92

Per-task values from runs.json. Task names are shortened for width. * scored from one judge.

Source hashes

Frozen artifact	SHA-256
v3.23/summary.json	e20a2d0627f8f3f890edbedf922acd3...
v3.23/protocol.json	a8ac6289f2fd0ac734d210f94f90be9e...
v3.23/private-manifest.json	d32c7b52a4d39ea1be6cceb51dbd822d...
v3.23/calibration-muse.json	bad50e05764d0926ec676d6d33b3a9da...
v3.23/calibration-grok.json	1401c7335f485508e46da9a3ed3bbf93...
comparison.json	79eaf3bb61e20ab17165bc018f37d2ab...
task-hash-bridge-sonnet55.json	544f591c01e85de46e6ee95b37192cf5...
sonnet55-v323-efforts.csv	2108e8d8ad61e3656bbf45e8d07b9281...
judge-pins-v3.json	da8e34dcdf771d05a02afb1ee67ba65f...
suite.lock.json	861da9e5386e88e6c77236ee5326863e...

VulcanBench Frontier v4: Claude Sonnet 5.5 across effort levels

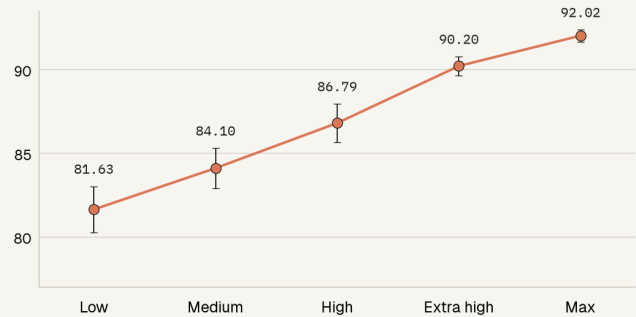
Combined score and runtime at every effort level, 23 tasks per effort. Code quality judged by Muse Spark 1.3 and Grok 4.6.

● Claude Sonnet 5.5 · Claude Code 2.1.291 to 2.1.293

n=23 at every effort, every run judged

Combined score

/100 · higher is better · focused scale



Mean runtime

Minutes per task · lower is better

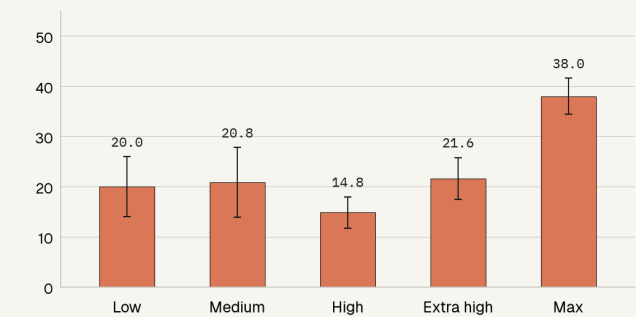


Table 1 | Code quality at each effort level

Component	Low	Medium	High	Extra high	Max
/100, mean of both judges					
Code quality	61.67	65.35	70.84	75.78	81.72
Human readability	53.4	54.4	65.9	72.2	79.5
Maintainability	64.9	68.7	75.1	78.0	84.1
Intent recovery	68.4	75.5	71.8	77.6	81.4
Rated by Muse Spark 1.3	56.1	60.4	69.7	74.4	81.5
Rated by Grok 4.6	62.2	62.7	71.3	75.8	82.2
Standard error of Code quality	2.08	1.66	1.57	1.28	0.99
Tasks passed (judged runs)	15	17	20	22	23
Runs with fallback-model turns	0	0	0	0	0
Cost per task, USD (list price)	2.88	3.00	2.39	3.38	5.52

Claude Code 2.1.291 for low (21 runs); 2.1.292 for low (2 runs), medium, high, extra high (22 runs); 2.1.293 for extra high (1 run), max. Earlier Claude columns used older CLIs. Refusal fallback on; no run used it. Cost is Claude Code's own list-price total per task, because the vendor price page gives two different Sonnet 5.5 cache-read prices. The sweep predates tagged run worktrees, so runs record no source commit and an older task hash; task content was verified identical to the suite lock and runs were admitted through a committed hash bridge. Judge settings match the original v3.3 and v3.4 protocols, recovered from a private backup after the judging host was erased. Muse ran the same binary (sha256 match); Grok ran through Cursor CLI 202610.01, which updates itself (v3.3 recorded 2026.09.02). Calibration passed with one allowed miss each: Muse Spark 1.3 missed g11 and Grok 4.6 missed g04. Frontier Code quality is L1 plus L2 and is not comparable with Routine v1 Code quality.

VulcanBench Frontier v4: Claude Sonnet 5.5, Opus 5.5 and Fable 5.1

Best combined score per model: Claude Sonnet 5.5 92.02 at max · Claude Fable 5.1 91.84 at max · Claude Opus 5.5 91.11 at high.

23 hard legacy-reconstruction tasks per effort level. Code quality judged by Muse Spark 1.3 and Grok 4.6.

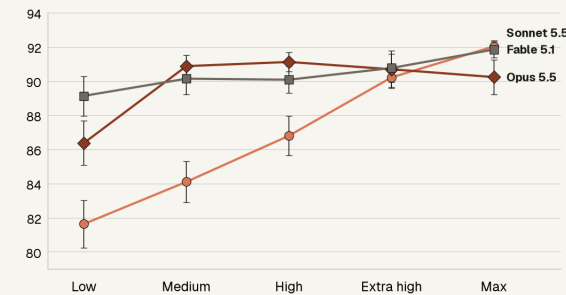
● Claude Sonnet 5.5 (with fallback)

◆ Claude Opus 5.5 (with fallback)

■ Claude Fable 5.1 (with fallback)

Combined score

/100 · higher is better · focused scale · whiskers +/-1 se



Cost per task

USD, API-equivalent at list prices · every serving model included · bases in notes

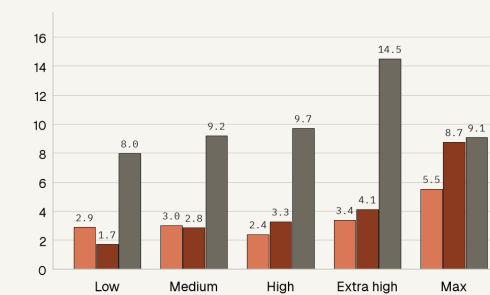


Table 1 | Every effort level

Model and measure	Low	Medium	High	Extra high	Max
● Claude Sonnet 5.5					
Combined score	81.63	84.10	86.79	90.20	92.02
Tasks passed of 23	15	17	20	22	23
Cost per task, USD	2.88	3.00	2.39	3.38	5.52
Fallback share of replies (runs)	0.0% (0)	0.0% (0)	0.0% (0)	0.0% (0)	0.0% (0)
◆ Claude Opus 5.5					
Combined score	86.36	90.86	91.11*	90.68	90.22
Tasks passed of 23	17	23	22	22	21
Cost per task, USD	1.70	2.84	3.27	4.11	8.75
Fallback share of replies (runs)	0.0% (0)	6.4% (3)	20.9% (7)	32.4% (8)	47.8% (12)
■ Claude Fable 5.1					
Combined score	89.12	90.13	90.07	90.75	91.84
Tasks passed of 23	19	20	20	22	23
Cost per task, USD	7.96	9.19	9.70	14.49	9.06
Fallback share of replies (runs)	6.9% (1)	34.1% (3)	23.9% (2)	40.3% (3)	12.3% (2)

* Combined score over the judged runs when fewer than 23. Bold marks each model's best level.

Protocols: Sonnet 5.5 is Code quality v3.23, Opus 5.5 is v3.15 and Fable 5.1 is v3.4. Same rubric, controls, weights and judges, judged in separate sessions. For v3.23 the judge settings were checked against the original protocols, recovered from a private backup: identical. Muse ran the same binary (sha256 match); Grok ran through Cursor CLI 2026.10.01, which updates itself (the v3.3 round recorded 2026.09.02).

All three ran with Claude Code's refusal fallback on and count every run; the share row counts replies written by another model. Sonnet 5.5 never fell back. Cost is Claude Code's own list-price total for Sonnet 5.5 and Opus 5.5, and the published cost ledger for Fable 5.1.

Harness versions differ (Sonnet 5.5 on 21.291 to 21.293, Opus 5.5 on 21.280, Fable 5.1 on 21.259 to 21.261), so small gaps between columns are harness confounded. Opus 5.5 and Fable 5.1 numbers are the published per-level rows.